File Input and Output (part 1)

CMSC 104 Section 2
November 4, 2026

Administrative Notes

Project 3 will be assigned next Monday, November 9

- The due date will be Monday, November 23

Quiz 3 is in class next Wednesday, November 11

Reading Data from files

To date, all input to all of our programs comes from the user typing at the keyboard

That's nice, and it's useful in a lot of situations, but it has limits

- If you have a large amount of data, you don't want to make the user type it in every time the program is run
- It makes it very difficult to analyze fairly large sets of data

So the solution is to allow data to be read in from existing files

Some constraints

In this class, we will only read data from “text” files

- Everything is a string
- The file can be plaintext (.txt); comma-separated value (.csv), tab-separated value (.tsv) or other format
- But everything’s a string
- And lines matter. Python recognizes the end-of-line character (“\n”) and uses it to control reading in files

Python can support reading in files in other formats - binary files containing graphics, and the like

- But we won’t cover those in this class

Some examples of files

- Examples of .txt file
- Examples of .csv file
- Examples of .tsv file

The options for reading in a Python data file

You can read in a ***string***, or a ***list of strings***

If you want to read in an entire file as a single string, Python has the “.read()” method

If you want to read in a single line as a single string, Python has the “.readline()” method - Python will read until the newline (“\n”) character and then stop

If you want to read in an entire file as a list of strings, separated at the newline character, Python has the “.readlines()” method

How it works

Python has a “file pointer” that indicates the next character to be read

- When you open the file for reading, this points to the first character in the file.
- When you use .readline() (singular; not plural) the file pointer moves to the next line

You can read in one or a few lines, and then use “.read()” or “.readlines()” to read the rest of the file.

The biggest issues:

Knowing the name of the file, including the directory path

Knowing the format of the data in the file

Steps

First, open the file:

- If you know the name and path ahead of time
- If you ask the user to enter the name

Next, read the contents

- readline() - read one line of the file as a string
- readlines() - read the rest of the file, one line at a time, as a list of strings
- read() - read the rest of the file, as a single string including all lines

Then, close the file

- The way we do it, this will be handled automatically

Opening the file

This is unique to Windows

If you know the name when you write the program

```
fname = "C:\\users\\aarsenault\\Documents\\Spring24.csv"
with open(fname, "r") as infile:
      data = infile.readline()
# now you have one line read in, and you can process it
      new_data = infile.readlines()
# now you have the rest of the file as a list of strings
```

Opening the file

There are many different ways in Python to open a file

- We recommend that you use “with open()” because that guarantees the file will be automatically closed when you’re done
- You don’t accidentally leave the file open and cause file system instability

Backslashes and escaping characters

If you want to encode “non-printable” characters in Python, you use a backslash

\ is a backslash; / is a slash

\n is a newline character

\t is a tab character

And so on.

Mac, Linux, etc. use forward slashes in their directory paths - so no problem

```
/alfredarsenault/PythonProjects/Data/Spring24.csv
```

But Windows uses backslashes - which can cause Python to misinterpret your path name as an escape character

- Spoiler alert: your program crashes, and you have to figure out why

Working through an example

See the file “Fall_2025.csv” on GitHub